评审材料

一、功能介绍

在企业研发与发布场景下，提供“代码注释自动转文档”和“一键部署打包”两项核心能力，解决文档滞后、发布链路不透明、人工操作易出错的问题。

功能价值如下：

- 在多人协作和频繁提交场景下，提供“提交即文档更新”的能力，解决接口说明依赖人工维护导致的失真问题。
- 在多环境发布和跨团队协同场景下，提供“可追踪、可回放、可中断”的一键部署能力，解决发布过程黑盒、回溯困难的问题。

MVP 范围与约束如下：

- 仅实现两个功能的简单版：代码注释转文档、一键部署打包。
- 不做登录界面，采用现有会话机制。
- GitLab 组在后端固定，不允许前端传入组名覆盖。
- GitLab Token 由后端接口写入 Session，后续 GitLab API 调用从 Session 读取。

二、架构及核心逻辑

2.1 整体技术栈与项目结构

系统采用 React 前端 + Xterm 中台 + Java Spring Boot 后端的混合技术栈架构，通过 pnpm workspace 管理所有 JS/TS 代码（前端+中台），Maven管理 Java 后端，两者在根目录下平行共存。

技术栈总览：

- 前端：React + TypeScript + Ant Design + Xterm.js + Vite + pnpm workspace umi CRA npm
- 后端：Java Spring Boot + Pty4J + Maven + Docker(待定) 免密 可选，目录注意

参考产品：打包部署功能参考 Vercel 的设计思路——关联 GitLab 仓库的交互模式、Build Log 的实时展示方式、部署流程的视觉反馈。

项目目录结构：

代码块

```
my_project/
```

```
2   ├── .github/                    # GitHub Actions CI/CD 配置
3   │   ├── workflows/
4   │   │   ├── frontend.yml        # 前端构建测试
5   │   │   ├── backend.yml         # 后端构建测试
6   │   │   └── e2e.yml             # 端到端测试
7   │   └── CODEOWNERS              # 代码归属权（后端/前端分开）
8   │
9   ├── apps/                       # 应用层（由 pnpm 管理）
10  │   ├── web/                    # React 前端应用
11  │   │   ├── src/
12  │   │   ├── public/
13  │   │   ├── package.json
14  │   │   ├── tsconfig.json
15  │   │   ├── vite.config.ts
16  │   │   └── index.html
17  │   │
18  │   └── terminal/               # Xterm 中台（独立应用）
19  │       ├── src/
20  │       │   ├── index.ts        # 导出 WebSocket 连接类
21  │       │   ├── xterm-manager.ts
22  │       │   └── types.ts
23  │       ├── package.json
24  │       ├── tsconfig.json
25  │       └── rollup.config.js
26  │
27  ├── packages/                   # 共享库（由 pnpm 管理）
28  │   ├── shared-types/           # TypeScript 类型定义（前后端共享）
29  │   │   ├── src/
30  │   │   │   ├── api.ts          # API 请求/响应类型
31  │   │   │   ├── websocket.ts    # WebSocket 消息类型
32  │   │   │   └── terminal.ts     # 终端相关类型
33  │   │   ├── package.json
34  │   │   └── tsconfig.json
35  │   │
36  │   ├── ui-components/          # React 组件库
37  │   │   ├── src/
38  │   │   ├── package.json
39  │   │   └── tsconfig.json
40  │   │
41  │   └── config-eslint/          # 共享 ESLint 配置
42  │       ├── package.json
43  │       └── index.js
44  │
45  ├── backend/                    # Java Spring Boot 后端（Maven管理）
46  │   ├── pom.xml                 # Maven 父 POM
47  │   ├── mvnw
48  │   ├── mvnw.cmd
```

```
49  │   │
50  │   ├── gateway-service/        # Spring Cloud Gateway（可选）
51  │   │   ├── pom.xml
52  │   │   └── src/
53  │   │
54  │   ├── terminal-service/       # 终端服务（WebSocket 处理）
55  │   │   ├── pom.xml
56  │   │   └── src/
57  │   │       ├── main/java/
58  │   │       └── main/resources/
59  │   │
60  │   └── business-service/       # 业务服务
61  │       ├── pom.xml
62  │       └── src/
63  │
64  ├── docker/                     # Docker 配置文件
65  │   ├── Dockerfile.frontend
66  │   ├── Dockerfile.terminal
67  │   ├── Dockerfile.backend
68  │   └── docker-compose.yml
69  │
70  ├── scripts/                    # 辅助脚本
71  │   ├── generate-api-types.sh   # 从 Java 注解生成 TS 类型
72  │   └── dev-init.sh
73  │
74  ├── docs/                       # 项目文档
75  │   ├── architecture.md
76  │   ├── api.md
77  │   └── development.md
78  │
79  ├── .gitignore
80  ├── .prettierrc
81  ├── .eslintrc.js
82  ├── pnpm-workspace.yaml         # pnpm workspace 配置
83  ├── package.json                # 根 package.json
84  ├── turbo.json                  # Turborepo 配置（可选）
85  ├── README.md
86  └── LICENSE
```

关键配置文件：

`pnpm-workspace.yaml`： 可以用shell脚本，分别打包前端后端

代码块

```
1  packages:
2    - 'apps/'
```

```
  - 'packages/'
  # backend 目录不在此列表，由 Maven 独立管理
```

根目录 `package.json` （统一命令入口）：

代码块

```
{
  "name": "my-project-monorepo",
  "private": true,
  "scripts": {
    "dev": "pnpm run dev:all",
    "dev:frontend": "pnpm --filter web dev",
    "dev:terminal": "pnpm --filter terminal dev",
    "dev:backend": "cd backend && ./mvnw spring-boot:run",
    "dev:all": "concurrently \"pnpm:dev:frontend\" \"pnpm:dev:terminal\" \"pnpm:dev:backend\"",
    "build": "pnpm run build:all",
    "build:frontend": "pnpm --filter web build",
    "build:terminal": "pnpm --filter terminal build",
    "build:backend": "cd backend && ./mvnw clean package",
    "build:all": "pnpm run build:frontend && pnpm run build:terminal && pnpm run build:backend",
    "test": "pnpm run test:frontend && pnpm run test:backend",
    "test:frontend": "pnpm --filter web test",
    "test:backend": "cd backend && ./mvnw test",
    "lint": "eslint apps packages --ext .ts,.tsx",
    "format": "prettier --write \"apps/**/*.{ts,tsx,css}\" \"packages/**/*.ts\""
  },
  "devDependencies": {
    "@types/node": "^20.0.0",
    "concurrently": "^8.0.0",
    "eslint": "^8.0.0",
    "prettier": "^3.0.0",
    "typescript": "^5.0.0"
  },
  "engines": {
    "node": ">=18.0.0",
    "pnpm": ">=8.0.0"
  }
}
```

`backend/pom.xml` （Maven 父 POM）：

代码块

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.myapp</groupId>
    <artifactId>myapp-backend</artifactId>
    <version>0.0.1</version>
    <packaging>pom</packaging>
    <modules>
        <module>gateway-service</module>
        <module>terminal-service</module>
        <module>business-service</module>
    </modules>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.0</version>
    </parent>
    <properties>
        <java.version>21</java.version>
    </properties>
</project>
```

2.2 架构图

后端架构图

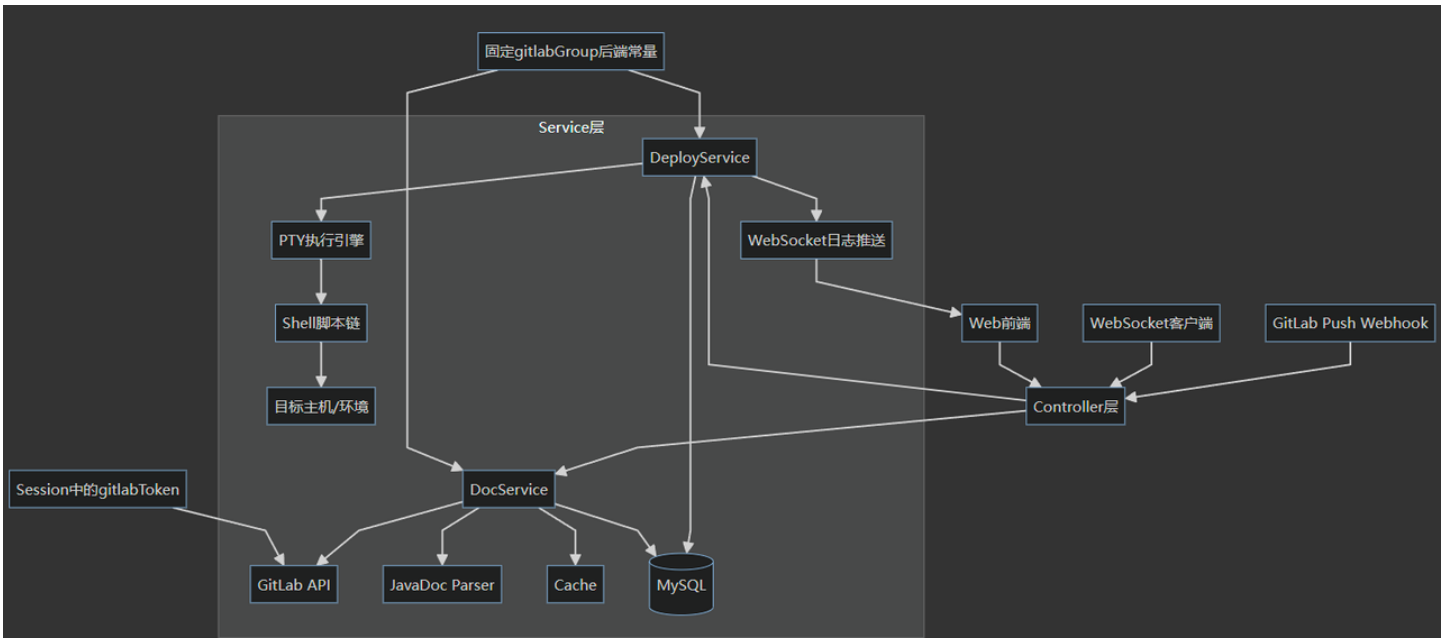
Webhook 私网配置

cache缓存 文档内存缓存 界面触发缓存失效 docker 另外部署mysql

1. Docker -> gitlab
2. Mysql
3. App
4. 打包 pack up

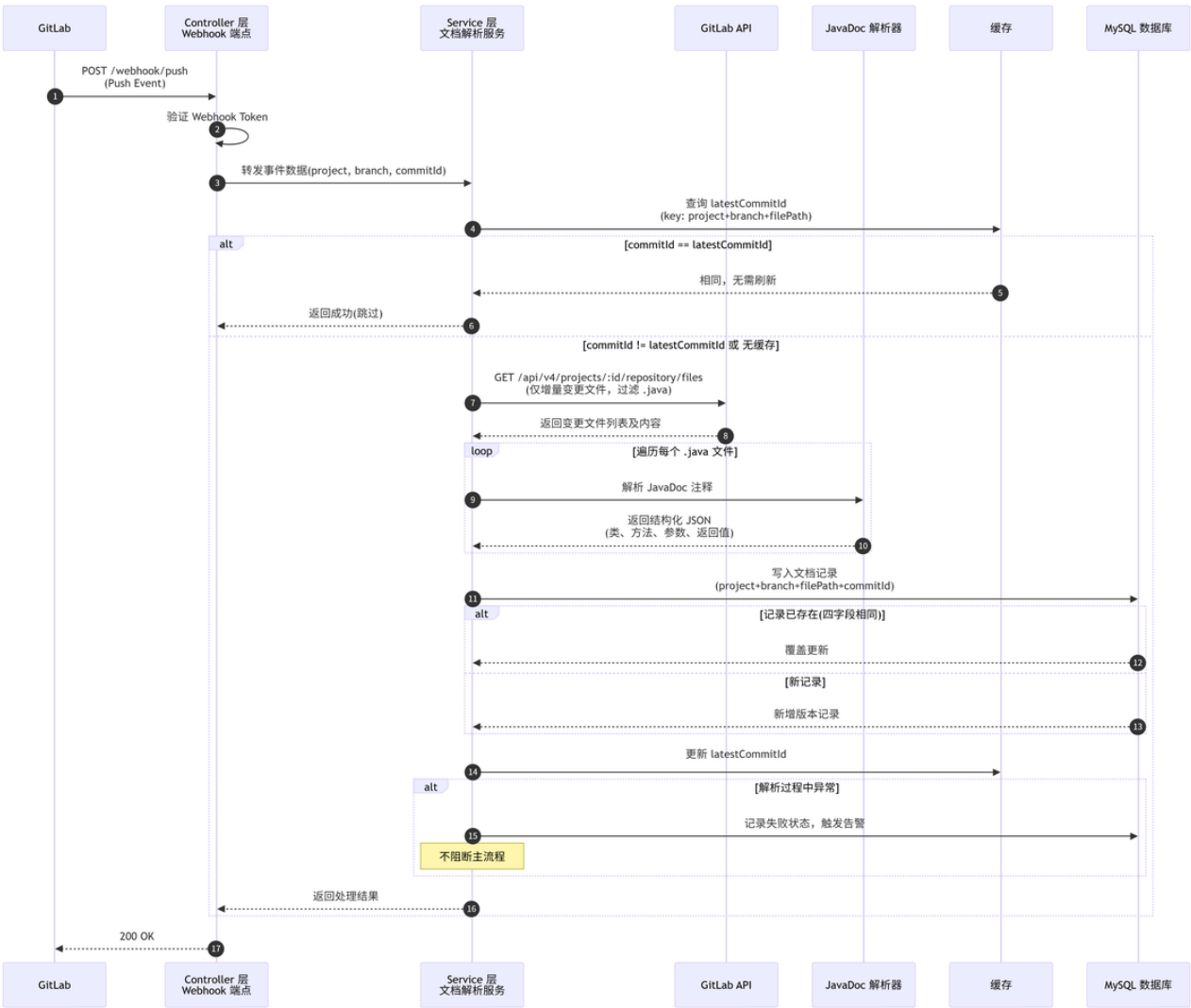
测试的时候，多docker把各个docker打包起来

win环境 wsl Linux 待定

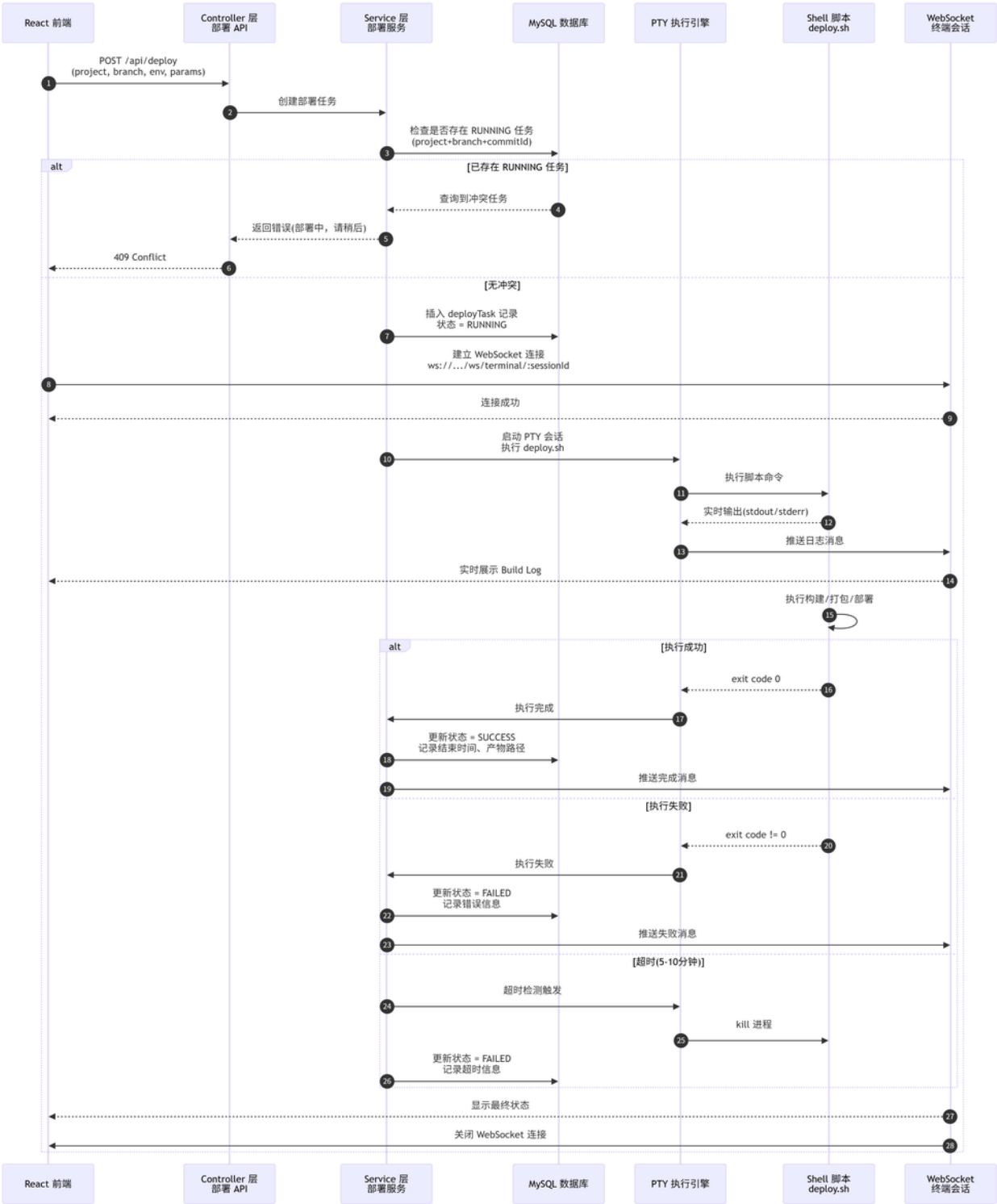


代码注释转文档时序图

单个接口画时序图，api设计，复杂的



一键部署打包时序图



2.3 总体架构

系统按照 SpringBoot 框架三层实现：

- **Controller 层**：HTTP、WebSocket
- **Service 层**：GitLab API、JavaDoc 解析器、脚本执行引擎（PTY + Shell）

- **Mapper 层**：MySQL、文件日志过程追踪

2.4 前端架构设计

`apps/web` （React 前端应用）：

代码块

```
{
  "name": "web",
  "version": "0.0.1",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc && vite build",
    "preview": "vite preview",
    "test": "vitest"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "@myapp/shared-types": "workspace:",
    "@myapp/terminal-client": "workspace:"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.0.0",
    "typescript": "^5.0.0",
    "vite": "^4.0.0"
  }
}
```

前端采用 React 18 + TypeScript + Vite 构建，通过 workspace 协议引用共享类型包和终端客户端包。

`apps/terminal` （Xterm 中台）：

代码块

```
{
  "name": "@myapp/terminal-client",
  "version": "0.0.1",
  "type": "module",
  "main": "./dist/index.js",
  "types": "./dist/index.d.ts",
  "exports": {
    ".": {
      "import": "./dist/index.js",
```

```
10          "types": "./dist/index.d.ts"
11        }
12      },
13      "scripts": {
14        "dev": "vite build --watch",
15        "build": "tsc && vite build",
16        "test": "vitest"
17      },
18      "dependencies": {
19        "xterm": "^5.3.0",
20        "xterm-addon-fit": "^0.8.0",
21        "xterm-addon-web-links": "^0.9.0"
22      },
23      "devDependencies": {
24        "typescript": "^5.0.0",
25        "vite": "^4.0.0"
26      }
27    }
```

终端中台封装 xterm.js 及 WebSocket 连接逻辑，作为独立库被前端引用。

`packages/shared-types` （共享类型定义）：

代码块

```
1   {
2     "name": "@myapp/shared-types",
3     "version": "0.0.1",
4     "type": "module",
5     "main": "./dist/index.js",
6     "types": "./dist/index.d.ts",
7     "scripts": {
8       "build": "tsc",
9       "dev": "tsc --watch"
10    },
11    "devDependencies": {
12      "typescript": "^5.0.0"
13    }
14  }
15  共享类型示例（api.ts）：
16  // 前后端共享的类型定义
17  export interface TerminalSession {
18    id: string;
19    createdAt: Date;
20    status: 'active' | 'closed';
21  }
22
```

```
export interface WebSocketMessage {
  type: 'stdin' | 'stdout' | 'resize';
  data: string | number[];
  sessionId: string;
}

export interface ApiResponse<T> {
  code: number;
  message: string;
  data: T;
}
```

前后端类型同步方案：通过脚本从 Java 代码生成 TypeScript 类型。

代码块

```
#!/bin/bash
# scripts/generate-api-types.sh
cd backend && ./mvnw spring-boot:run -Dspring-boot.run.arguments=--openapi.export=/tmp/openapi.json
npx openapi-typescript /tmp/openapi.json -o packages/shared-types/src/generated/api.ts
```

2.5 功能 A：代码注释自动转在线文档

问题：每次 commit 后，如何把 Java 注释自动转换为结构化在线文档？怎么追溯到 commit？

技术：

- GitLab API（拉增量文件）
- JavaDoc 解析（类、方法、param、return）
- MySQL

逻辑：

1. GitLab 克隆仓库到达后端
2. 提取 project、branch、commitId，过滤仅处理 Java 文件变更（默认规则：filePath 以 `.java` 结尾，配置文件 java py 分类写好，yaml 等类似配置）
3. 调用 GitLab API 拉取变更文件内容（仅增量，避免全仓再次克隆）
4. 调用 JavaDoc 解析器
5. 根据结构化的 JSON 提取结构体（类、方法、注释摘要、参数、返回值）
6. 以 project + branch + filePath + commitId 写入数据库
7. 先比对 commitId 与缓存中的 latestCommitId；相同则不刷新，不同才刷新

8. 对外提供查询 API（按项目、分支、文件、commit 查询）

关键设计：

- 每条文档记录保留 commitId 与 author，支持比较和回滚
- 解析失败不阻断主流程，落失败状态并告警

多语言扩展细节：先看后缀，`.java` 就先走 Javadoc；后续多语言在后端放一个 Map（后缀 → 工具），没有命中的文件写警告信息，不直接报错中断。

去重与刷新策略：同一个 eventId 或 commitId 可能被重复推送，后端做去重——查到该 commit 已经处理过，直接返回成功，不再重复解析。缓存刷新策略：先比对 commitId，和缓存里 latestCommitId 一样就不刷，不一样才刷新，避免重复计算。文档入库按 project + branch + filePath + commitId 判断：四个值一样就覆盖更新，不一样就新增一条版本记录。

2.6 功能 B：一键部署打包

问题：如何让用户在 Web 端一键触发打包/部署，并实时看到执行状态与日志。

技术：

- WebSocket（实时日志推送）
- PTY 终端执行（后端执行脚本并回传）
- Shell 脚本链（构建、打包、部署）
- GitLab API（必要时获取分支/提交信息）
- MySQL

前端终端集成示例：

代码块

```
// apps/web/src/hooks/useTerminal.ts
import { TerminalClient } from '@myapp/terminal-client';
import type { WebSocketMessage } from '@myapp/shared-types';

export const useTerminal = (sessionId: string) => {
  const ws = new WebSocket(`ws://localhost:8080/ws/terminal/${sessionId}`);
  const client = new TerminalClient(ws);

  client.onMessage((msg: WebSocketMessage) => {
    if (msg.type === 'stdout') {
      // 处理终端输出
    }
  });

  return client;
};
```

逻辑：

1. 前端调用“触发部署”API，传入项目、分支、环境、部署参数
2. 后端创建 deployTask 记录，状态置为 Running
3. 后端通过 PTY 启动部署脚本（deploy.sh） 进程并行 内存
4. 执行输出推送到 WebSocket，前端实时展示日志
5. 脚本完成后更新任务状态为 SUCCESS/FAILED，并记录结束时间
6. 如果有产物路径/下载信息，回写 result 字段供后续下载或外传

关键设计：

- 状态转换：RUNNING → SUCCESS/FAILED
- **防重复部署**：同一 project + branch + commitId 同时仅允许一个 RUNNING 任务，避免互相覆盖环境
- **超时保护**：每个任务设置超时（如 5 分钟或 10 分钟），超时后自动标记 FAILED，并尝试 kill 执行进程，防止一直占资源

Pty ssh远程连接，可并行，任务完成状态可以依据日志解析

2.7 当前版本约束

- GitLab Group 固定为后端常量，不从前端透传
- Session 中保存 gitlabToken，调用 GitLab API 时优先取 Session Token
- 无登录界面

三、存储设计

3.1 数据库选型与职责

- **MySQL**：任务主数据、文档索引、审计信息

3.2 核心表设计

> MVP 先不考虑日志功能，后续还要加上日志相关的表

核心表：

日志

3.2.1 文档任务表（doc_task）

用于记录代码注释转文档的任务流水和状态。

字段名	类型	默认值	允许NULL	描述
task_id	bigint		否	任务主键 ID
event_id	varchar(64)	''	否	GitLab 推送事件唯一标识
project_name	varchar(64)	''	否	项目名称
branch_name	varchar(64)	''	否	分支名称
commit_id	varchar(64)	''	否	代码提交版本号
run_status	tinyint	0	否	运行状态：0-等待，1-执行中，2-成功，3-失败
duration_second	int	0	否	任务耗时（秒）
create_time	datetime		是	任务创建时间

3.2.2 文档明细表（doc_file_dtl）

存储具体的类、方法等结构化文档内容，作为文档的主表明细 。

字段名	类型	默认值	允许NULL	描述
detail_id	bigint		否	明细主键 ID
project_name	varchar(64)	''	否	项目名称
branch_name	varchar(64)	''	否	分支名称
file_path	varchar(256)	''	否	Java 文件完整路径
commit_id	varchar(64)	''	否	对应的提交版本号
doc_content	text		是	文档结构的 JSON 字符串
update_time	datetime		是	最后更新时间

3.2.3 部署任务表（deploy_task）

用于记录一键部署打包的任务流水。

字段名	类型	默认值	允许NULL	描述
task_id	bigint		否	部署任务主键 ID
project_name	varchar(64)	''	否	项目名称
branch_name	varchar(64)	''	否	分支名称
script_name	varchar(64)	''	否	执行的 Shell 脚本名称
script_arg	varchar(256)	''	否	脚本执行参数
run_status	tinyint	0	否	状态：0-运行中，1-成功，2-失败
operator_name	varchar(32)	''	否	触发操作人姓名
duration_second	int	0	否	部署持续时间（秒）
create_time	datetime		是	任务创建时间（起始点）
log_path	varchar(256)	''	否	部署日志存储路径

3.2.4 构建任务表（build_task）

用于记录代码编译、镜像构建等构建环节的任务流水。

字段名	类型	默认值	允许NULL	描述
task_id	bigint		否	构建任务主键 ID
project_name	varchar(64)	''	否	项目名称
branch_name	varchar(64)	''	否	分支名称
commit_id	varchar(64)	''	否	待构建的代码提交版本号
build_type	varchar(32)	''	否	构建类型：如 jar, docker, npm 等
run_status	tinyint	0	否	状态：0-进行中，1-成功，2-失败，3-已取消
operator_name	varchar(32)	''	否	触发构建的操作人
duration_second	int	0	否	构建持续时间（秒）
create_time	datetime		是	任务创建时间（起始点）
output_path	varchar(256)	''	否	构建产物或镜像的存储路径

索引：

- **文档唯一约束：** `uk_project_branch_file_commit`
  - 组成：`project_name, branch_name, file_path, commit_id`
  - 作用：防止同一版本的同文件重复入库。
  -
  -
- **部署查询索引：** `idx_project_status_time`
  - 组成：`project_name, run_status, update_time`
  - 作用：优化按项目查看部署历史的性能。

数据库设计规范 https://bbs.huaweicloud.com/blogs/409870

3.3 大表处理

> MVP 先不考虑，后续的 docFile 和 deployLog 可能成为数据很多的大表

四、API 设计

4.1 对外 API

文档流水线：

- `POST /api/session/setGitlabToken` 获取用户token
- `POST /api/doc/rebuild` 刷新操作
- `GET /api/doc/query` 查询已存在的文档

部署流水线：

- `POST /api/deploy/trigger` 触发打包
- `GET /api/deploy/task` 部署任务状态
- `GET /api/deploy/log` 日志
- 取消部署 websocket会关闭连接，不需要设计API

4.2 鉴权与数据隔离模型

MVP 鉴权：

- 不做登录界面
- 通过 Session 维持 token 与操作上下文
- 接口使用签名密钥校验来源

数据隔离：

- 此阶段先在后端固定 GitLab 组

4.3 限流与防重

- 触发部署：同一 commitId 同时仅允许一个 RUNNING 任务
- 防重复：eventId/commitId 去重

4.4 命令行交互

- 后端统一通过脚本执行器调用 Shell 命令
- 标准化返回：exitCode、stdout/stderr 摘要

五、开发工作流与 CI/CD

5.1 本地开发初始化

代码块

```
# 1. 克隆项目
git clone <your-repo>

# 2. 安装前端依赖
pnpm install

# 3. 构建共享包
pnpm --filter @myapp/shared-types build

# 4. 安装后端依赖
cd backend && ./mvnw dependency:resolve

# 5. 启动所有服务（一键启动）
pnpm dev
```

5.2 CI/CD 配置（GitHub Actions）

代码块

```yaml
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]

jobs:
  frontend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v2
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'pnpm'
      - run: pnpm install
      - run: pnpm run lint
      - run: pnpm run test:frontend
      - run: pnpm run build:frontend

  backend:
    runs-on: ubuntu-latest
    steps:
```

```
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v3
        with:
          java-version: '21'
          distribution: 'temurin'
          cache: 'maven'
      - run: cd backend && ./mvnw verify
```

5.3 Docker 编排

代码块

```yaml
# docker/docker-compose.yml
version: '3.8'
services:
  backend:
    build:
      context: ../backend
      dockerfile: ../docker/Dockerfile.backend
    ports:
      - "8080:8080"
    environment:
      - SPRING_PROFILES_ACTIVE=docker

  frontend:
    build:
      context: ..
      dockerfile: docker/Dockerfile.frontend
    ports:
      - "3000:3000"
    depends_on:
      - backend

  terminal:
    build:
      context: ..
      dockerfile: docker/Dockerfile.terminal
    ports:
      - "3001:3001"
```

六、监控/报警/应急设计

6.1 稳定性风险识别

> ⚠ 存在稳定性风险

主要风险：

- 任务阻塞：脚本卡死导致 RUNNING 长时间不结束
- 解析异常：异常 Java 文件导致文档解析失败
- 外部依赖波动：GitLab API 超时

6.2 监控指标

核心指标：

- `docPipelineSuccessRate`
- `docParseLatency`
- `deploySuccessRate`
- `deployRunningDuration`
- `gitlabApiErrorRate`

6.3 报警策略

- 部署任务 RUNNING 超过 5 分钟报警 实测一下
- 文档解析超过 5 分钟报警
- GitLab API 连续超时报警

6.4 应急预案

- 任务恢复：对失败任务支持按 taskId 重试
- API 异常重试：GitLab API 异常时切到重试队列

七、性能评估

7.1 查询性能影响与优化

影响点：文档查询可能按项目/分支/路径高频访问

优化策略：文档主查询命中索引（project + branch + filePath + commitId）

7.2 入库性能影响与优化

影响点：单次 push 可能包含多个 Java 文件，造成突发写入

优化策略：文档入库批量写

7.3 MVP 性能目标

缓存加速gitlab获取仓库文件

指标	目标值
docPipelineLatencyMs（单次 commit ≤20 个 Java 文件）	≤ 5000ms
docQueryLatencyCacheHitMs（命中缓存）	≤ 200ms
docQueryLatencyCacheMissMs（未命中缓存）	≤ 800ms
deployTriggerApiLatencyMs（触发部署 API 同步返回）	≤ 300ms

八、成本评估

V2.0版本需求明确和原型设计，预计1周
V2.0版本研发详细方案设计和评审，预计1周
编写代码完成<在线文档查看>需求和<自动打包部署>需求，预计4~5周
完成V2.0版本测试，输出测试报告，预计1周

预计 4 - 5 周完成开发任务 。

每周四周会同步进度，当前困难或者疑惑沟通（**注意会前自己准备好，提高沟通效率**）。

按照周会第二天作为新的一个工作周期。

4.17 - 4.24

4.24 - 5.1

一周一个P0，两个P0完成，基本可用，开始测试 / 迭代

5.1 - 5.8（含五一假期）兜底，这个时候如果P0都不能用只能假期继续做了

5.8 - 5.15

5.15 - 5.22

8.1 里程碑与协作规范

迭代策略：

1. 优先完成在线文档查看，再扩展自动打包部署
2. MVP 阶段只做单文件处理，在线文档查看对应文件执行 docs 生成脚本
3. 任务按个人认领，后续会明确开始时间、截止时间，**定期同步进度，如果延期了要更改及时和团队沟通！！！一定要沟通！！！**

协作注意事项：

- **注意commit规范，完成小功能马上推！！！防止merge conflict 太多，直接merge ，不用rebase ，如果有污染代码的担忧，不确定的功能添加测试自己切分支走pr。一般情况下直接推main主干开发。commit注意格式，前缀：描述，英文**
- AI 生成的代码必须 Review，避免循环定义、类型守卫滥用、any 类型等问题
- 在自己能力范围内关注代码质量
- 优先查阅官方文档和源码，借助 AI 辅助理解
- 小问题优先组内尝试解决，提高协作效率
- 遇到困难及时沟通，可按里程碑约定短会（约 10 分钟）同步交流

排期说明：按里程碑设定 DDL，通过短会对齐进度。

鼓励机制：鼓励每位成员尝试全栈开发，不要畏惧学习的心智负担，前后端任务可根据自身情况灵活选择。

UI 说明：原型图如有疑问及时沟通确认。

⭐任务清单

https://applink.feishu.cn/client/todo/task_list?guid=8e193e2d-80e6-47d1-b822-546fd50ae0cc

---

8.2 在线文档查看模块（优先级 P0）4.17 - 4.23

任务	说明	负责人	开始时
首页 Dashboard	卡片布局，预留扩展位置，登录组件占位	杨凌	2026/4,
虚拟终端初始化	Xterm 初始化 + WebSocket 连接，注意逻辑复用	杨凌	2026/4,
建表，定义结构体，尝试从结构体写入数据库	设计文档、仓库、用户等表结构，ORM 映射并测试写入	陈弘博	2026/4,
后端终端对接前端，跑通一个简单命令	WebSocket 消息路由，执行 echo/ls 等基础命令并返回	陈弘博	2026/4,
后端实现gitlab仓库克隆到本地	调用 git clone，支持 SSH/HTTPS，处理鉴权	陈弘博	2026/4,
前端交互	Button 唤起虚拟终端，连接 GitLab API	杨凌	2026/4,
实现判断后缀 + 调用对应文档生成工具逻辑	根据文件后缀（.java/.py/.md 等）调用 javadoc、pydoc 等工具	陈嘉毅	2026/4,
把 javadoc 生成的文档转换为结构体	解析 javadoc XML/HTML，提取类、方法、字段等存入结构体	陈嘉毅	2026/4,
实现新 commit 判断逻辑，提取新的文档	Webhook 监听或轮询，识别新增/修改的 commit，触发文档提取	张俊杰	2026/4,
实现 commit 和旧 commit 比对逻辑	对比两次 commit 的差异，定位变更文件，增量更新文档	张俊杰	2026/4,
第三方登录	Session Cookie 加密，或尝试 GitLab OAuth 登录	杨凌	2026/4,
登录登出	注意安全性、鉴权，支持第三方登录（GitLab）	杨凌	2026/4,
后端在线部署	配置 GitLab CI/CD 流水线，自动生成并部署文档站点	陈弘博	2026/4,
数据持久化	前端 IndexedDB / LocalStorage 缓存 ，刷新页面后恢复状态	杨凌	2026/4,
文档查看	双栏布局：左侧列表（支持 CRUD），右侧查看（MVP 只读，后续迭代支持编辑）	杨凌	2026/4,

> **里程碑**：完成上述任务后，在线文档查看模块基本可用

8.3 在线文档查看优化需求（优先级 P1）

任务	说明
文档分享链接	支持导出文档
数据存储	数据库存储，或浏览器存储（IndexedDB）作为离线方案
在线编辑权限	代码支持本地 Web 在线 CRUD
注释自动补全/代码审查	调用大模型 API，增加 Button 或浮窗助手，流式输出需选用合适 SDK 或自行实现
历史记录	支持撤销/重做（Undo/Redo）
批量生成	多文件同时处理（难度较高，后续实现）

8.4 自动打包部署模块（优先级 P0）4.24 - 4.29

任务	说明	负责人	开始时间	结
shell 脚本编写	编写部署/构建所需的 Shell 脚本，支持环境初始化、依赖安装、构建产物生成等	陈嘉毅、张俊杰	2026/4/23	20
修改 Pty 逻辑，CMD/部署脚本	将单路伪终端拆分为两路，分别支持 CMD 命令执行和部署脚本运行	陈嘉毅	2026/4/23	20
写触发部署的接口	提供 REST API 触发部署任务，接收请求后异步执行，返回任务状态（Running/Success/Failed）	张俊杰	2026/4/23	20
页面搭建	前端页面框架搭建，包括布局、路由、组件基础结构	杨凌	2026/4/24	20
部署日志收集与展示	部署过程中实时收集 stdout/stderr，支持前端 WebSocket 流式查看，失败时可回溯日志	杨凌	2026/4/24	20
实现 CommitID 冲突时的逻辑	确保同一个 CommitID 同时只能解析一次，使用分布式锁或数据库唯一约束防止重复解析	陈嘉毅	2026/4/27	20

8.5 自动打包部署模块（优先级 P1）

P1的内容待补充

任务	说明	负责人	开始时间	结束时间
部署历史记录	记录每次部署的 CommitID、触发人、开始/结束时间、状态，支持查看最近 10 条	待定	待定	待定
通知机制	部署成功/失败时发送飞书/钉钉消息通知	待定	待定	待定
并发部署锁	同一个项目同时只能跑一个部署任务，防止资源冲突	待定	待定	待定
部署超时控制	设置部署任务最大执行时间（如 30 分钟），超时自动终止并标记失败	待定	待定	待定
环境变量管理	支持项目级别的环境变量配置，部署时注入到脚本中（如数据库密码、API Key）	待定	待定	待定

8.6 基础功能迭代（优先级 P2）

任务	说明
多语言支持	不仅限于 JavaDoc，支持批量上传多语言文件
前端性能优化	长列表渲染、性能压力测试
后端数据安全	数据库加密、防御性措施
容器化部署	上线 Docker

8.7 进阶功能探索（优先级 P3）

AI 能力集成

功能	实现思路
构建错误智能诊断	拦截 PTY 输出流中的 stderr 或非零退出码，截取最后 50-100 行报错日志，结合 Commit 信息调用大模型分析，返回修复建议并展示在前端终端旁
遗留代码一键注释补全	文档提取时检测核心函数缺失注释，UI 提供“AI 补全”按钮。后端读取函数源码送大型生成符合规范（如 Javadoc、JSDoc）的注释，通过 GitLab API 反向提交回代码库
Codebase RAG	将提取的文档、函数说明向量化（Embedding），存入轻量级向量数据库（如 Chroma、pgvector）。前端提供对话框，AI 检索代码库返回参数使用示例

架构优化

优化项	实现思路
LRU 缓存	后端实现哈希表+双向链表的 LRU 缓存，拦截 GitLab API 获取仓库信息、高频文件读取的请求。未命中时查询远端，命中时移动节点到链表头部，超出容量淘汰尾节点
配置变更 MVCC / WAL	在线编辑配置时采用多版本并发控制（MVCC）记录历史版本，或实现简易预写日志（WAL）：Commit 到 GitLab 前先将变更操作写入后端日志，崩溃后可根据日志恢复

系统进阶拓展

能力	实现思路
Tree-sitter 万能语法树解析	统一引入 Tree-sitter（支持多语言的极速解析库），用统一代码实现 C、Java、JS、Python 的方法和注释提取逻辑
Docker in Docker 隔离编译	程序运行在容器中，通过挂载宿主机 Docker Socket 动态启动干净的编译容器（装配特定版本 Java/Node 镜像），构建完成即销毁，实现环境隔离

其他设想

- 构建个人知识库 Agent（RAG），类似飞书知识库
- 自研 SDK 等等

> **说明**：迭代过程中可适当融入 github trending 的技术（如 AI、RAG 等），但需确保核心产品功能可用。

九、后端核心能力清单

能力	说明
GitLab API 对接	仓库信息获取、文件拉取、Commit 提交等
多语言文档解析	针对不同语言采用对应工具（如 Java → JavaDoc）
WebSocket + PTY 实时构建推送	建立 WebSocket 通信，使用 PTY 将子进程输出实时转换为流推送前端
SSH 远程部署	编写 Bash 脚本实现远程部署